SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:40**

Annexure A

Industry Problem Solving Task

Code of Conduct:

I R. Durga Sree/192411189 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. DURGA SREE

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyse how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

Industry Problem Understanding

Banking must coordinate accounts, fund transfers, loans, bill payments, transaction history, and fraud monitoring. Its key constraints are confidentiality, integrity, auditability, transactional consistency, availability, and extensibility.

Object-Oriented Principles

Encapsulation protects balances, loan state, payment details, and transaction information.

Abstraction lets services use Account, Loan, Payment, and Transaction contracts instead of concrete implementations.

Polymorphism supports savings, current, business, and specialized account types.

Composition models relationships such as Customer–Account and Account–Transaction.

High cohesion and single responsibility separate transfer processing, fraud detection, loan calculation, and persistence.

Design Patterns

Factory Pattern creates the correct account type without exposing construction logic. New account types can be localized.

Strategy Pattern encapsulates loan-interest and payment-calculation policies. New regulatory or product-specific policies can be added without rewriting the loan service.
Observer Pattern publishes transaction events to fraud-alert, notification, and audit observers. Repository Pattern provides domain-oriented persistence and hides OODBMS-specific details.

Layer-to-OODBMS / Processing Flow

During a fund transfer, the presentation layer sends a request to the transfer service. The service loads source and destination Account objects through repositories, checks authorization and business constraints, performs the domain operations, records a Transaction, and commits the transaction. The OODBMS persists the updated accounts and transaction relationships. Observers can then receive the transaction event. Atomic transaction handling ensures that a debit cannot be persisted without its corresponding credit.

Scalability, Flexibility, and Maintainability

Scalability comes from extension points for account types, policies, and alert consumers. Security is strengthened by layered authorization, protected domain state, transaction boundaries, audit events, and least-privilege persistence. Maintainability improves because replacing persistence technology does not require rewriting the domain model.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.
The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Industry Problem Understanding

Retail must maintain accurate products, suppliers, customers, purchase orders, sales, and warehouse stock while supporting changing product categories and high transaction volumes.

Object-Oriented Principles

Encapsulation protects stock quantities, reorder levels, pricing, and order states.

Polymorphism supports category-specific product behavior through common contracts.

Composition models PurchaseOrder–Supplier, Sale–Customer, and Warehouse–InventoryItem relationships.

Interfaces make business services testable with mock DAOs and future prediction services.

Design Patterns

MVC separates models, views, and controllers so the UI can evolve independently. Factory Pattern centralizes product-category creation.

Observer Pattern allows stock changes to notify managers or procurement systems. DAO Pattern isolates CRUD and query operations for persistent inventory objects. **Layer-to-OODBMS / Processing Flow**

A sale reaches a business service through an MVC controller. The service retrieves Product and InventoryItem objects through the DAO, validates availability, creates the Sale, updates stock, and commits the transaction. The OODBMS persists the changed objects and relationships. A stock event can trigger an Observer when inventory falls below the minimum threshold.

Scalability, Flexibility, and Maintainability

Automated stock prediction can be introduced as a new service consuming historical inventory data, while robotic warehouse management can consume domain events or APIs. Indexing SKU, supplier, warehouse, and stock status, together with transaction-safe stock updates and pagination, supports performance.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify

communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyse how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Industry Problem Understanding

Hotel operations combine guest registration, reservations, room availability, check-in/check-out, payments, room services, and feedback. The system must respond to changing occupancy and apply flexible pricing.

Object-Oriented Principles

Encapsulation protects room availability, reservation status, payment state, and service charges.

Polymorphism supports different Room and Service types.

Composition connects Guest, Reservation, Room, Payment, and ServiceOrder objects.

Abstraction allows pricing and notification components to depend on interfaces.

Design Patterns

Strategy Pattern supports seasonal, loyalty, promotional, and other pricing policies. Factory Pattern creates room and service objects according to configured types.

Observer Pattern notifies guests, housekeeping, staff, and dashboards about reservation or service events.

Repository Pattern provides operations such as findAvailableRooms(), saveReservation(), and findGuestHistory().

Layer-to-OODBMS / Processing Flow

The presentation layer submits reservation details to the service layer. The reservation service validates guest information and dates, queries the RoomRepository, selects a PricingStrategy, creates or updates Reservation objects, and persists the changes. The repository interacts with the OODBMS to retrieve room objects and store reservations and relationships. After commit, observers can send confirmations or update operational dashboards.

Scalability, Flexibility, and Maintainability

New pricing strategies, room types, and notification consumers can be added without rewriting the reservation workflow. Concurrency controls are essential to prevent double

booking; indexes on room status, type, reservation dates, and guest identifiers improve retrieval.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

Industry Problem Understanding

Logistics must manage shipments, warehouses, vehicle allocation, delivery tracking, and customer notifications while coping with changing routes and real-time events.

Object-Oriented Principles

Encapsulation protects shipment state, vehicle allocation, route data, and delivery status.

Polymorphism supports different shipment types with shared contracts.

Composition represents Shipment–Package, Vehicle–Assignment, and Delivery–TrackingEvent relationships.

Interfaces permit replacement of route optimizers, notification providers, and DAOs.

Design Patterns

Strategy Pattern selects routes according to distance, cost, traffic, priority, or environmental constraints.

Factory Pattern creates standard, express, refrigerated, oversized, or other shipment types.

Observer Pattern distributes real-time tracking events to customers, operations staff, and dashboards.

DAO Pattern isolates persistence for Shipment, Vehicle, TrackingEvent, and allocation objects.

Layer-to-OODBMS / Processing Flow

Shipment registration enters through the presentation layer and is processed by a business service. The service validates the shipment, uses the Factory to construct it, invokes a routing Strategy when required, and asks the DAO to persist shipment and tracking information. The OODBMS stores persistent objects and relationships. Delivery status changes update tracking objects and trigger notification observers.

Scalability, Flexibility, and Maintainability

AI route optimization can be introduced as another Strategy or external service. Autonomous vehicles can be integrated through new vehicle abstractions and adapters. Indexing shipment, vehicle, status, warehouse, and timestamp fields plus concurrency control for vehicle allocation supports scale and reliability.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

Industry Problem Understanding

Energy management monitors smart meters and consumption, analyzes usage, and generates abnormal-consumption alerts. It must ingest frequent readings while remaining extensible for renewable-energy monitoring and AI prediction.

Object-Oriented Principles

Encapsulation protects meter state, readings, thresholds, and energy rules.

Polymorphism supports different smart-meter types and capabilities.

Composition links Meter, EnergyReading, ConsumptionProfile, Alert, and AnalysisResult objects.

Abstraction isolates analysis algorithms and persistence operations behind interfaces.

Design Patterns

Observer Pattern sends consumption or anomaly events to users, operators, dashboards, and alerting services.

Strategy Pattern supports alternative energy-optimization algorithms.

Factory Pattern creates different smart-meter objects.

Repository Pattern provides domain-oriented persistence for meters, readings, alerts, and analysis results.

Layer-to-OODBMS / Processing Flow

Meter data enters through the presentation/API layer and is handled by the business layer. The domain layer validates readings and applies energy rules. The EnergyRepository stores Meter and EnergyReading objects in the OODBMS. For analysis, the business layer retrieves the required time range through the repository, applies an optimization Strategy, and stores results or alerts. Abnormal readings can publish events without coupling domain objects to notification infrastructure.

Scalability, Flexibility, and Maintainability

Renewable-energy classes such as SolarPanel, BatteryStorage, and GenerationReading can be added without redesigning existing layers. AI prediction can be a new Strategy or service. Indexes on meter identifiers and timestamps, batching or streaming ingestion, historical archiving, and separate real-time alert processing improve scalability.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, welldocumented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation